

생성형 AI(LLM)와 인간 : 텍스트 판별 챌린지

2025 SW중심대학 디지털 경진대회 : AI부문

인하대학교 TEAM 에이아이들
이제동 김솔 김유현 소윤희 한상민

INTRO

대회 소개



0

Human Generated



1

AI Generated

주어진 문단(paragraph)을
사람이 작성 vs 생성형 AI가 작성
구분하는 **이진 분류 문제**

모델은 각 문단에 대해 AI가 작성했을 확률(0~1)을 예측해야 함

데이터 구성

TRAIN DATA

단위 : 위키피디아 전체 글
(full_text)

구성 : title, full_text,
generated (0 or 1)

full_text에 대한 라벨만 제공
→ 일부 문단만 AI가 작성한 경우도
generated = 1

TEST DATA

단위: 문단 (paragraph)

구성: ID, title,
paragraph_index,
paragraph_text

문단 단위로 확률 예측 필요

Train 데이터에서 0:1 비율은 약 11:1로 구성되어 있음

데이터 길이 분포

TRAIN DATA 길이 분포 분석

[Original **full_text** 길이 분포]

	count	mean	std	min	25%	50%	75%	max
generated								
0	89177.0	2325.40	3351.24	624.0	926.0	1331.0	2339.0	98549.0
1	7995.0	2298.66	3131.24	393.0	918.0	1334.0	2301.0	46814.0

[**문단 단위**로 잘랐을 때 길이 분포]

	count	mean	std	min	25%	50%	75%	max
generated								
0	1125599.0	179.6	151.8	3.0	75.0	146.0	243.0	19114.0
1	100710.0	180.4	138.4	7.0	84.0	150.0	241.0	4001.0

데이터의 특성

[문단 단위 라벨 부재]

◆ 학습 데이터는 전체 글(Full Text)에 대해, 일부 문단이나 문장만 AI가 작성된 경우에도 글 전체에 'AI 작성' 라벨(1)이 부여되며, 문단 단위 라벨은 제공되지 않습니다.

DACON '문제' 일부

Train data의 full_text(1)를 문단 단위로 분할 시 각 문단의 0,1 라벨 혼동

도메인 일반화

AI detector는 다양한 도메인의 문단에 대해 견고한 예측 성능을 요함



주제 / 스타일

실제 사용 시 입력 문단의
주제나 스타일은 미지



OUT-OF-DISTRIBUTION

따라서 train data와는 다른 도메인
(OOD)에서도 잘 작동해야 함



일반화 성능

일반화 성능이 모델의
실제 활용 가능성을 좌우

데이터 구성

TRAIN DATA

위키피디아 기반
→ 인물, 과학, 문화 등 **다양한 주제**

but, 백과사전 스타일
→ 비교적 **일관된 문체**

TEST/실무 DATA

주제와 문체가 명확하지 않음

train과 도메인 차이 발생 가능성 존재
(out-of-distribution)

따라서 모델이 **unseen domain에서도 성능을 유지**하려면
domain generalization 전략 필요

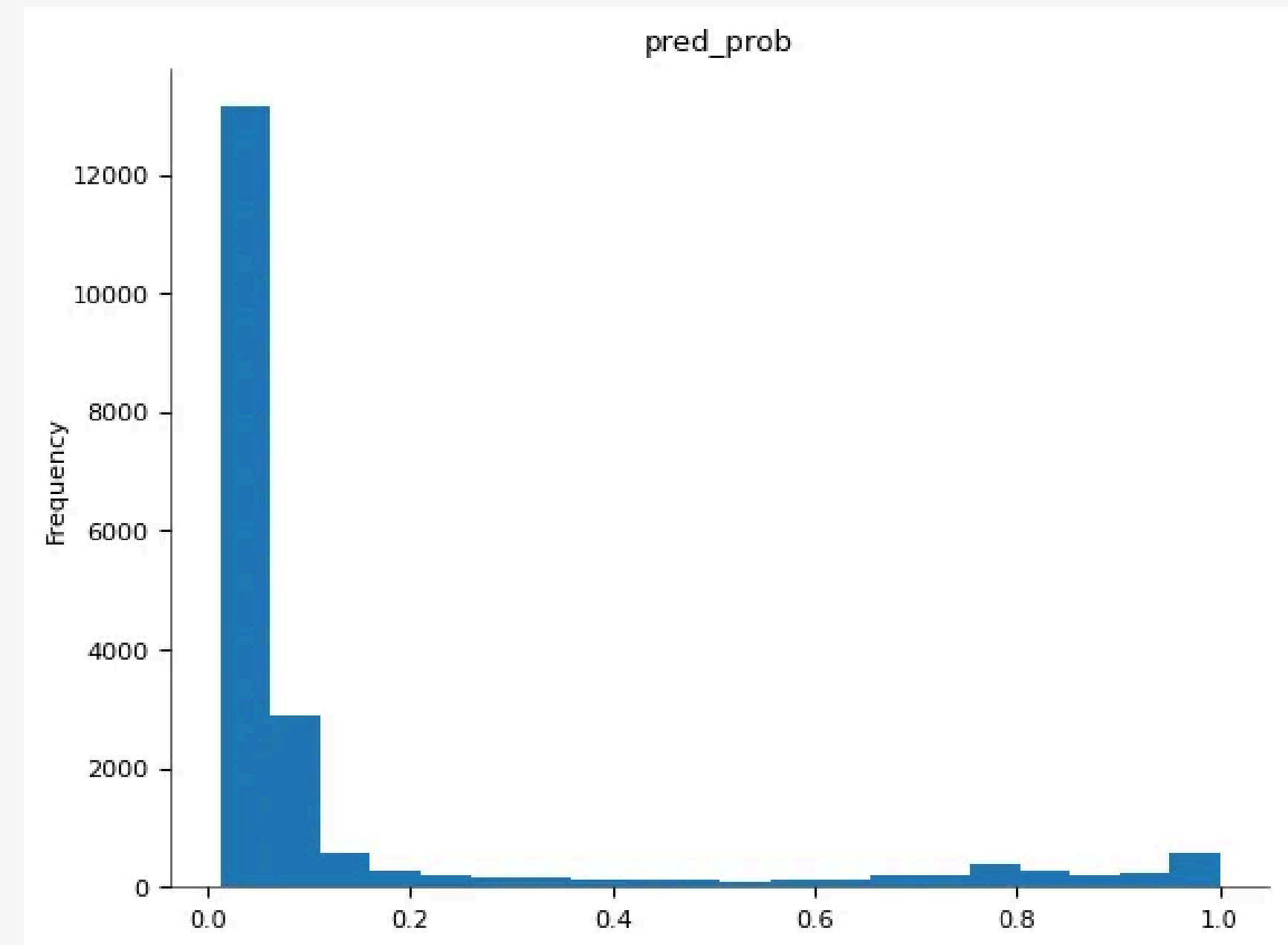
데이터셋 비율은 어떻게 맞춰야할까?

학습은 원래 데이터 라벨 비율 (약 11:1) 그대로 학습

validation dataset을 0,1 레이블 비율 1:1로 구성

0번으로 몰린 추론 → Validation AUC: 0.7285

결론: 라벨비를 맞추어 학습해야 한다! → Under Sampling



노이즈가 포함되어도 괜찮을까?

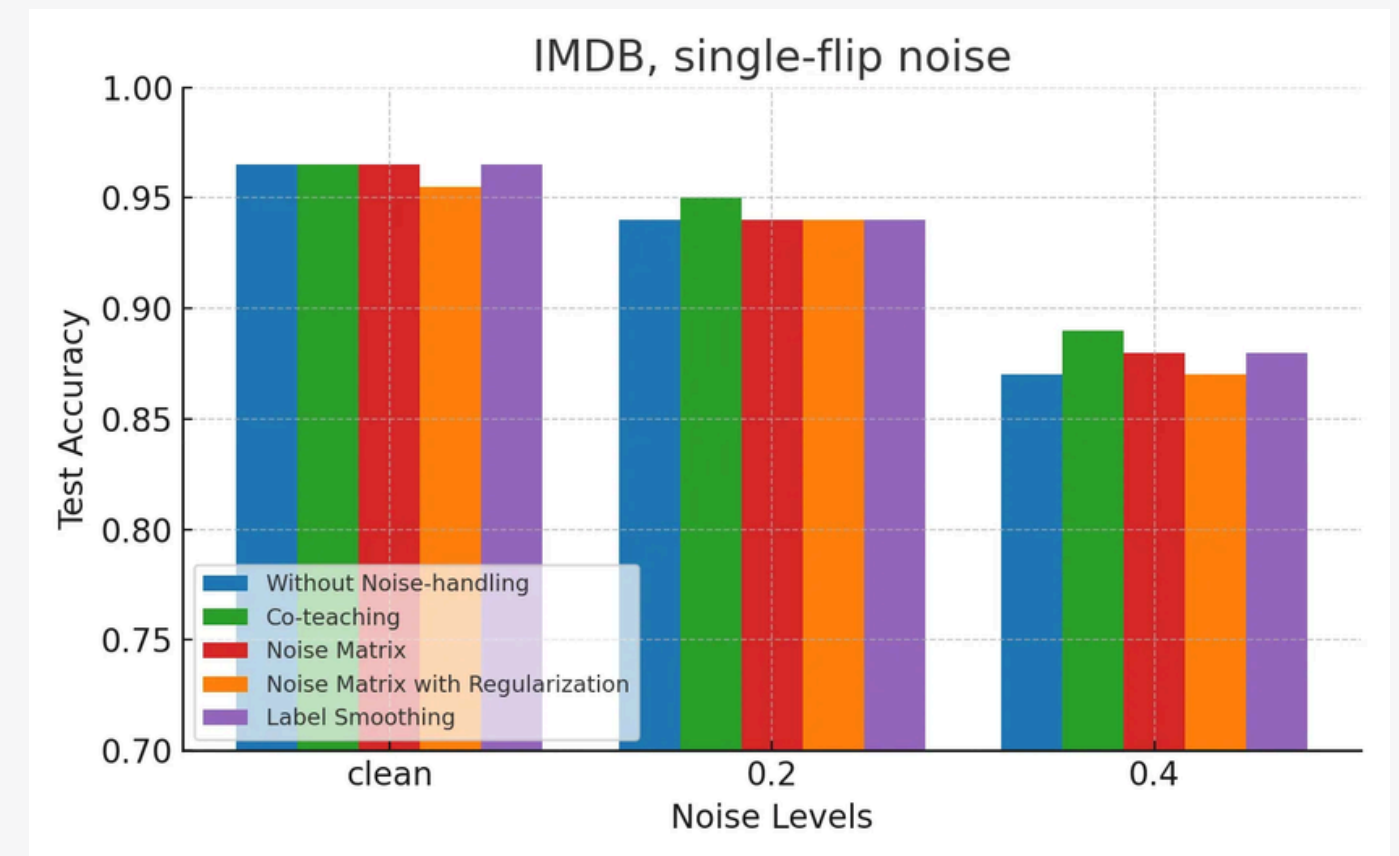
IMDB · single-flip(편향) 노이즈 실험에서 노이즈율을
최대 40%까지 올려도 노이즈 처리 전과 후의 격차가 4%p 이내

⇒ 노이즈가 상당히 존재해도,
특별한 노이즈 교정 기법이 가져오는 실질적 이득은 미미
(복잡도·추가 비용 대비 효과가 크지 않음)

동일한 Transformer 계열에서

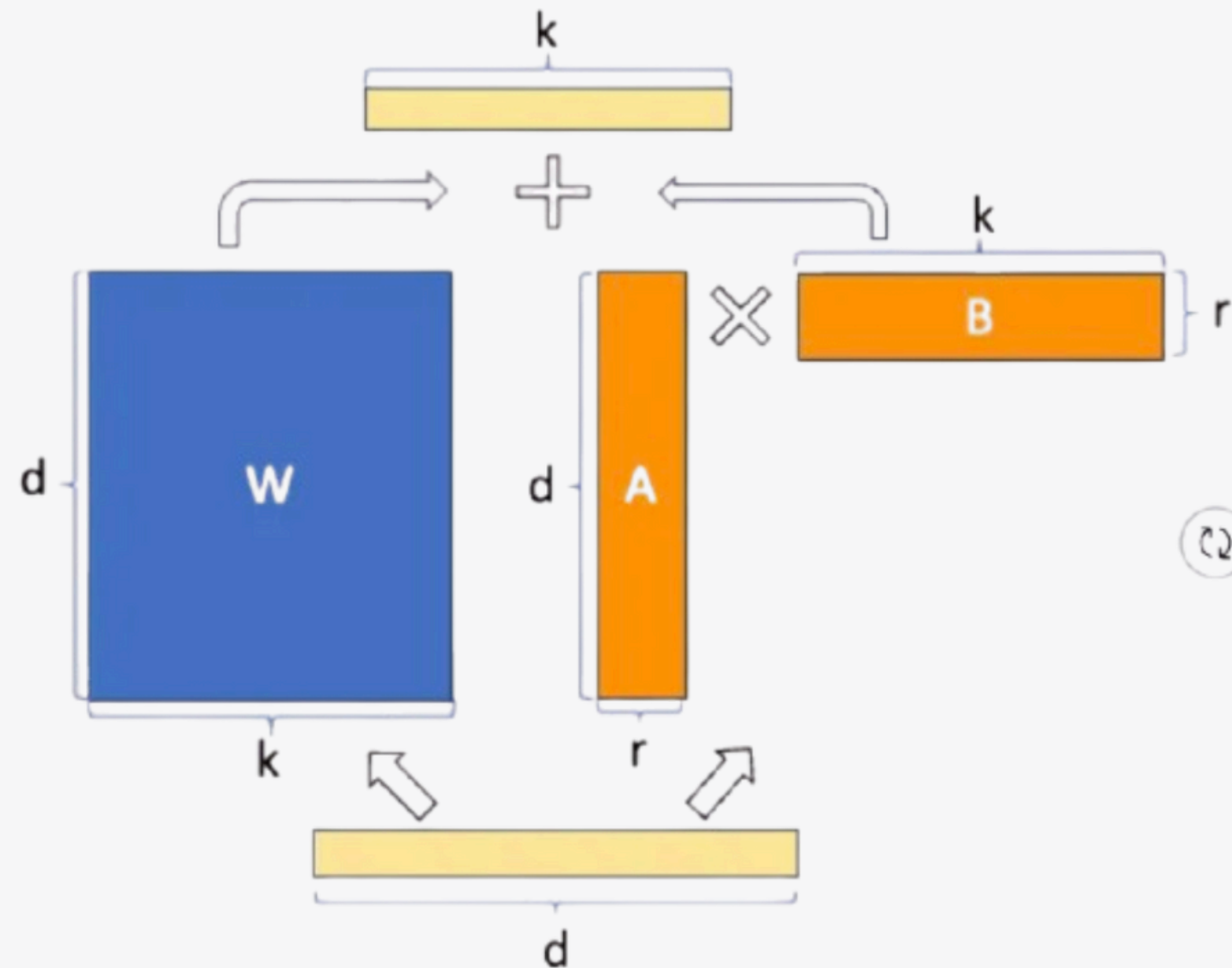
1. full_text(1)을 문단 단위 분할
2. 분할된 문단들을 모두 1로 라벨링(재라벨링X)

→ 대규모 성능 저하 위험은 제한적



Zhu et al., Is BERT Robust to Label Noise? A Study on Learning with Noisy Labels in Text Classification, ACL Insights 2022.

LORA



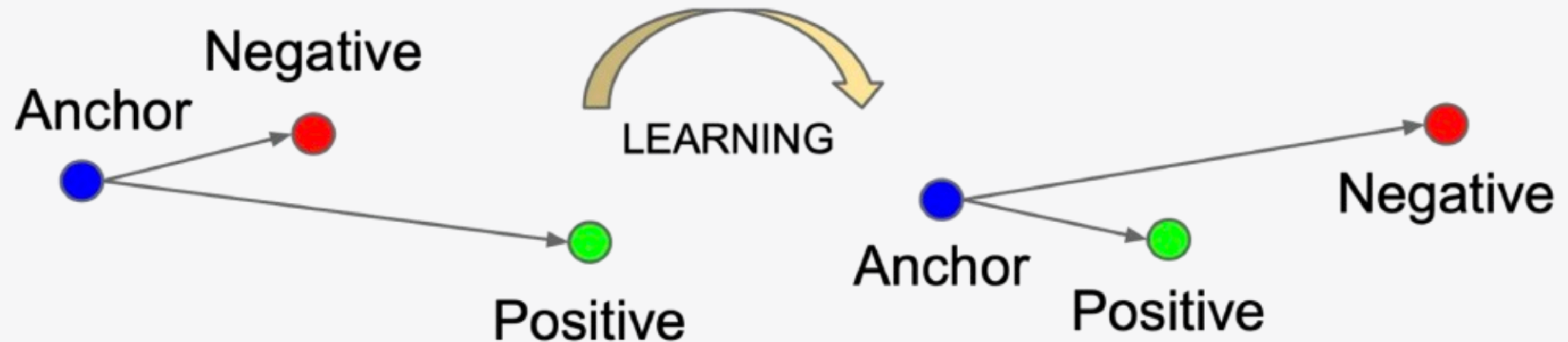
domain generalization을 위해 모델의 사전지식 활용

즉 작은 모델을 full-fine-tuning시, domain generalization 불가

따라서 큰 모델을 Lora fine-tuning : 기존 모델의 약 1%의 파라미터만 수정

= LoRA 파인튜닝

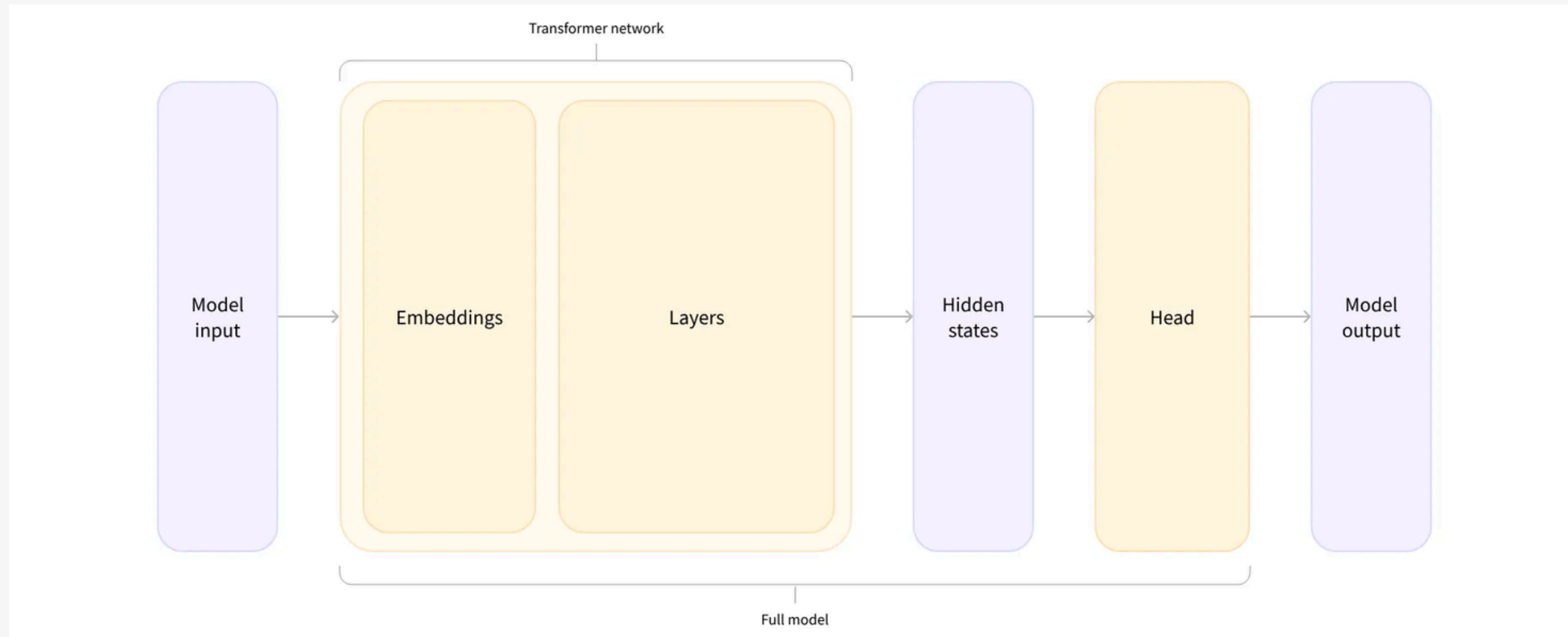
CL LOSS



보편적인 이진 분류 loss는 CE Loss이지만, CE는 눈 앞의 domain만 잘 학습
→ domain generalization이 안됨

→ Contrastive Loss 도입

CLASSIFICATION HEAD



대부분의 encoder 모델보다 **text generation 모델**이 훨씬 더 많은 **사전 지식** 보유
but, text generation 모델은 **decoder only** 구조이기 때문에 detection task와 맞지 않음

→ Classification Head 장착

BASE MODEL SELECTION

선택한 BASE LLM 모델

GOOGLE/GEMMA3

91.69

KAKAO/KANANA

93.38

QWEN/QWEN3

91.31

LGAI/EXAONE

93.94

STACKING ENSEMBLE

4가지 모델의 4fold의 예측 확률 병합 → 새로운 학습 데이터(meta_train) 생성
→ meta model(LR) 학습

94.21

Stacking Ensemble

1개 fold 당 4개 base 모델 학습 : 13H
4 fold : 52H

fold를 이용해 전체 train data로 meta model 학습 가능
but, 학습시간 4배 소요

BLENDING ENSEMBLE

고정된 validation data 예측 확률로만 meta model(LR) 학습

94.39

Blending Ensemble

4개 모델 학습 : 13H

학습시간 4배 단축

실제 문제에 얼마나 적용할 수 있을까?

적용가능성



LLM 모델 변경

KT/midm
SKT/A.X-4.0
LGAI/EXAONE-4.0

&



CLASSIFICATION HEAD

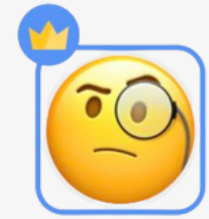
official
customization

다른 LLM 모델로 detection task 수행 가능

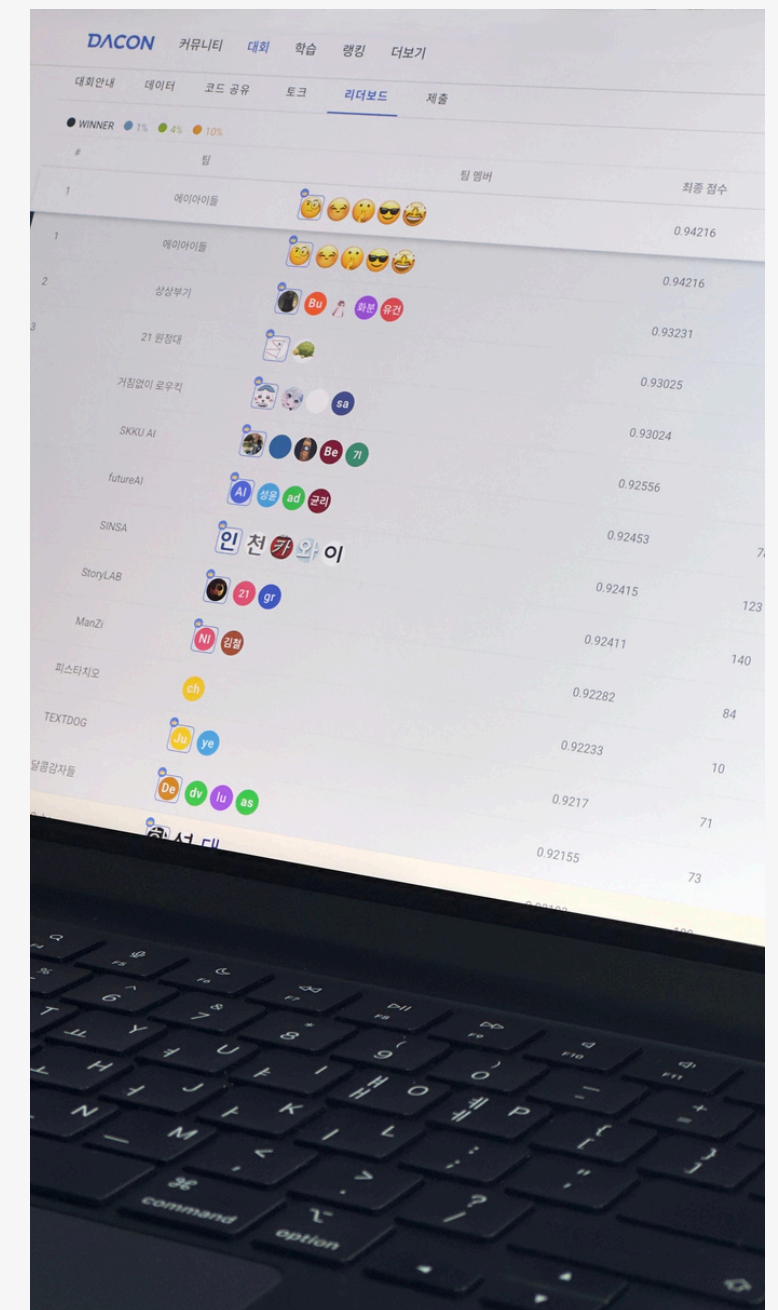
LLM 기반이지만, **quantization + LoRA로 경량화** → 실무 적용 용이

Q&A

AI-DLE



진짜진짜진짜마지막최종데이터전처리0714-4폴드.ipynb의 사본



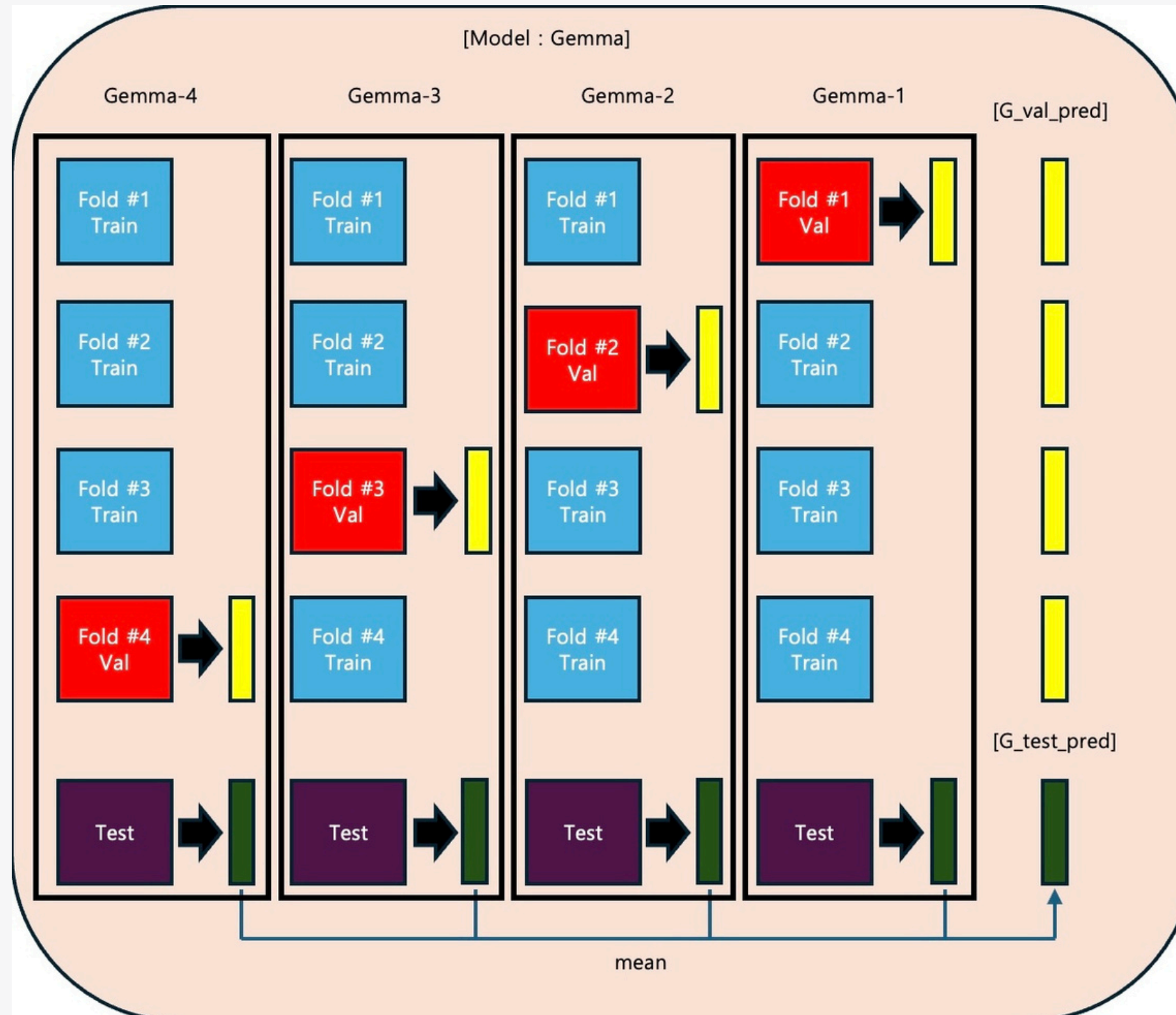
2025-digital-gemma-3-InfoNCE백만버그수정스케줄링렐루느낌3090.ipynb

부록

APPENDIX

4-FOLD STACKING ENSEMBLE

STEP1 : 4-FOLD 한 데이터로 BASE MODEL 학습 및 추론



다음과 같은 과정을 각 BASE MODEL (GEMMA, QWEN, KANANA, EXAONE)에 대해서 진행

1. 모델 당 4 fold 서로 다른 데이터로 학습한 **4개의 분기** 생성
2. 분기별로 val에 대한 추론 결과를 stack 하여 **Gemma_val_pred** 생성
3. 분기별로 test 데이터를 추론한 결과를 평균내어 **Gemma_test_pred** 생성
4. 4개의 모델에 대해서 같은 방식으로 진행하고
{model}_val_pred를 병합하여 **meta_train** 생성
{model}_test_pred를 병합하여 **meta_test** 생성

4-FOLD STACKING ENSEMBLE

STEP2 : {MODEL}_VAL_PRED 병합 → META_TRAIN 생성
{MODEL}_TEST_PRED 병합 → META_TEST 생성

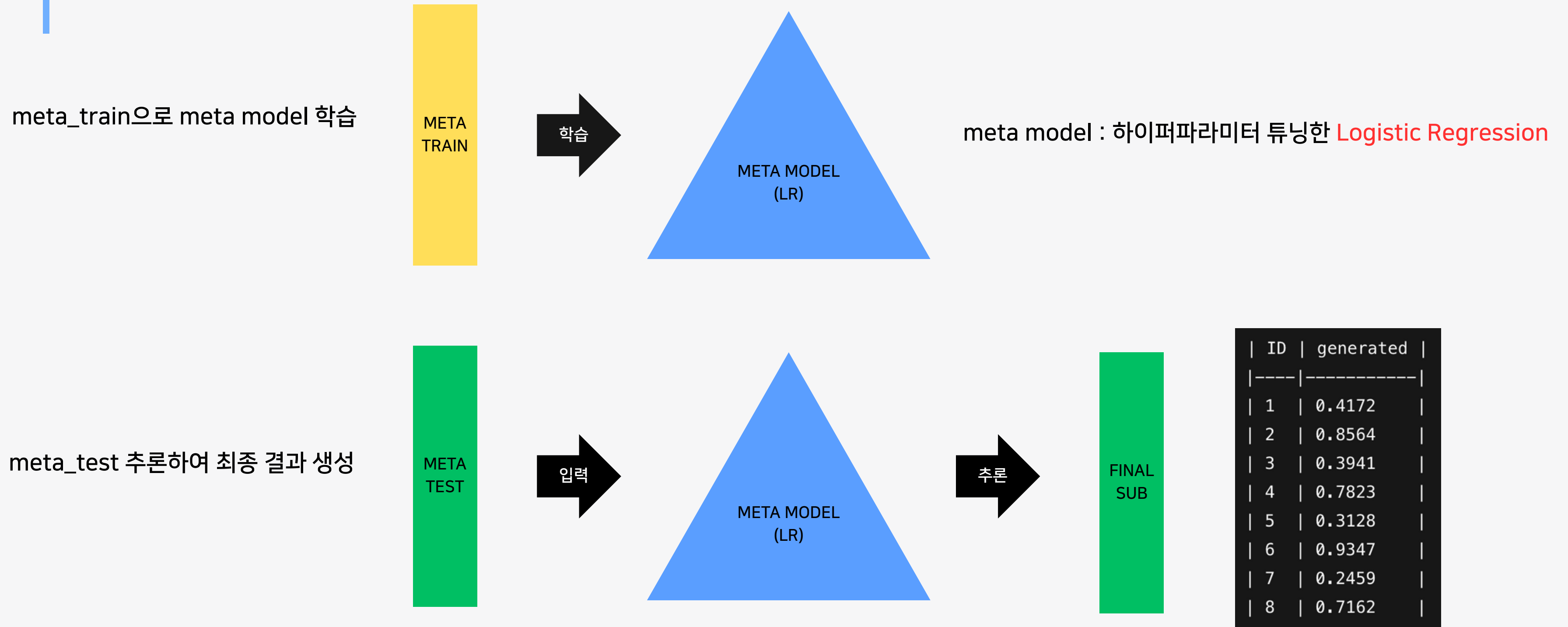
[meta_train]

ID	G_val_pred	Q_val_pred	K_val_pred	X_val_pred	Ground-Truth label
1	0.12	0.23	0.08	0.15	0
2	0.87	0.91	0.79	0.85	1
3	0.34	0.29	0.41	0.38	0
4	0.76	0.68	0.71	0.80	1
5	0.44	0.55	0.49	0.51	0
6	0.93	0.89	0.94	0.90	1
7	0.28	0.35	0.31	0.29	0
8	0.81	0.75	0.77	0.84	1
9	0.22	0.18	0.25	0.19	0
10	0.66	0.72	0.70	0.69	1

[meta_test]

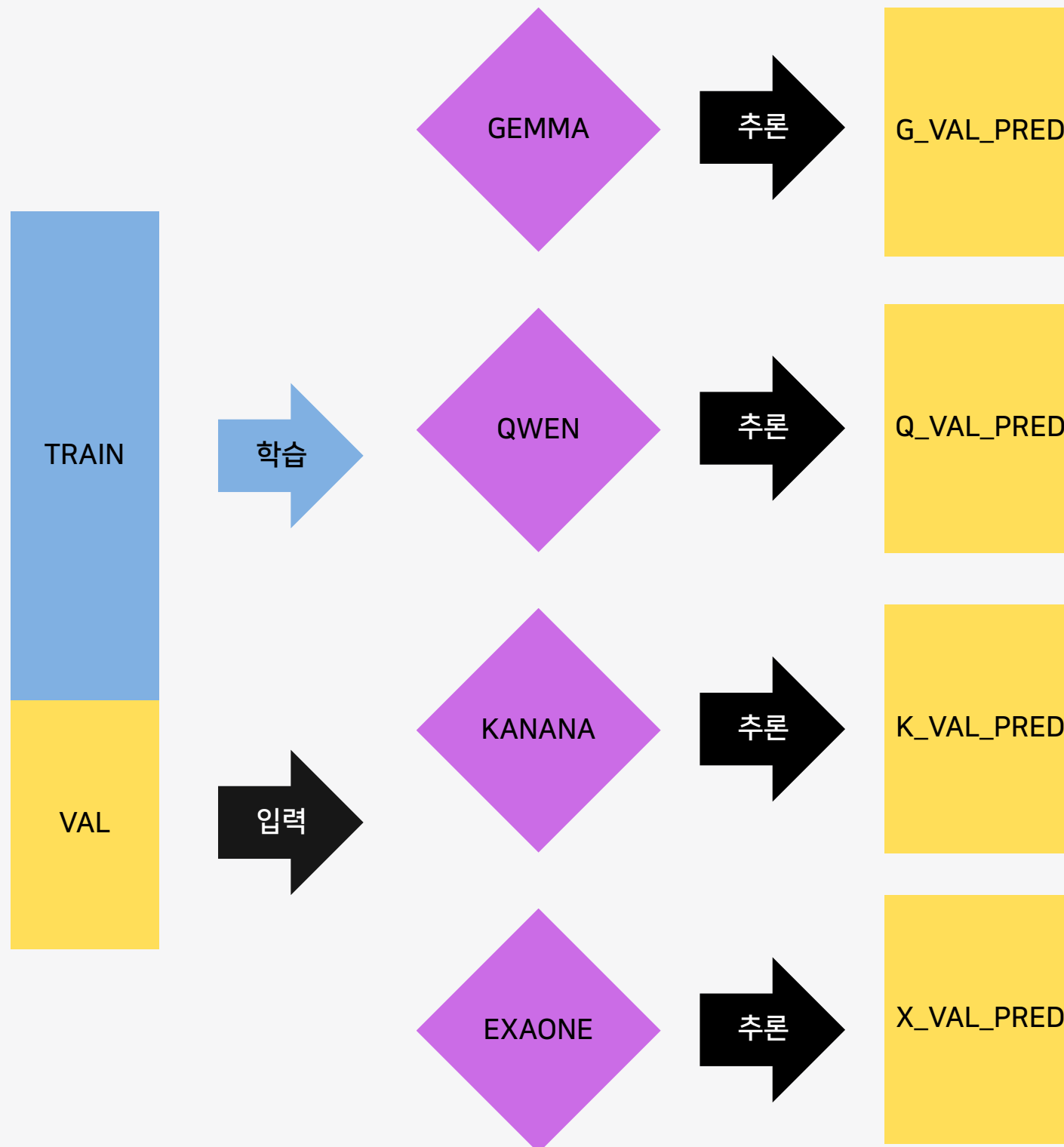
ID	G_test_pred	Q_test_pred	K_test_pred	X_test_pred
1	0.41	0.36	0.29	0.33
2	0.78	0.82	0.76	0.80
3	0.52	0.47	0.50	0.49
4	0.11	0.09	0.13	0.10
5	0.66	0.70	0.69	0.68
6	0.25	0.30	0.27	0.26
7	0.91	0.89	0.94	0.92
8	0.38	0.34	0.36	0.35
9	0.59	0.63	0.61	0.60
10	0.73	0.77	0.75	0.74

STEP3 : META MODEL 학습 및 추론



4-MODEL BLENDING ENSEMBLE

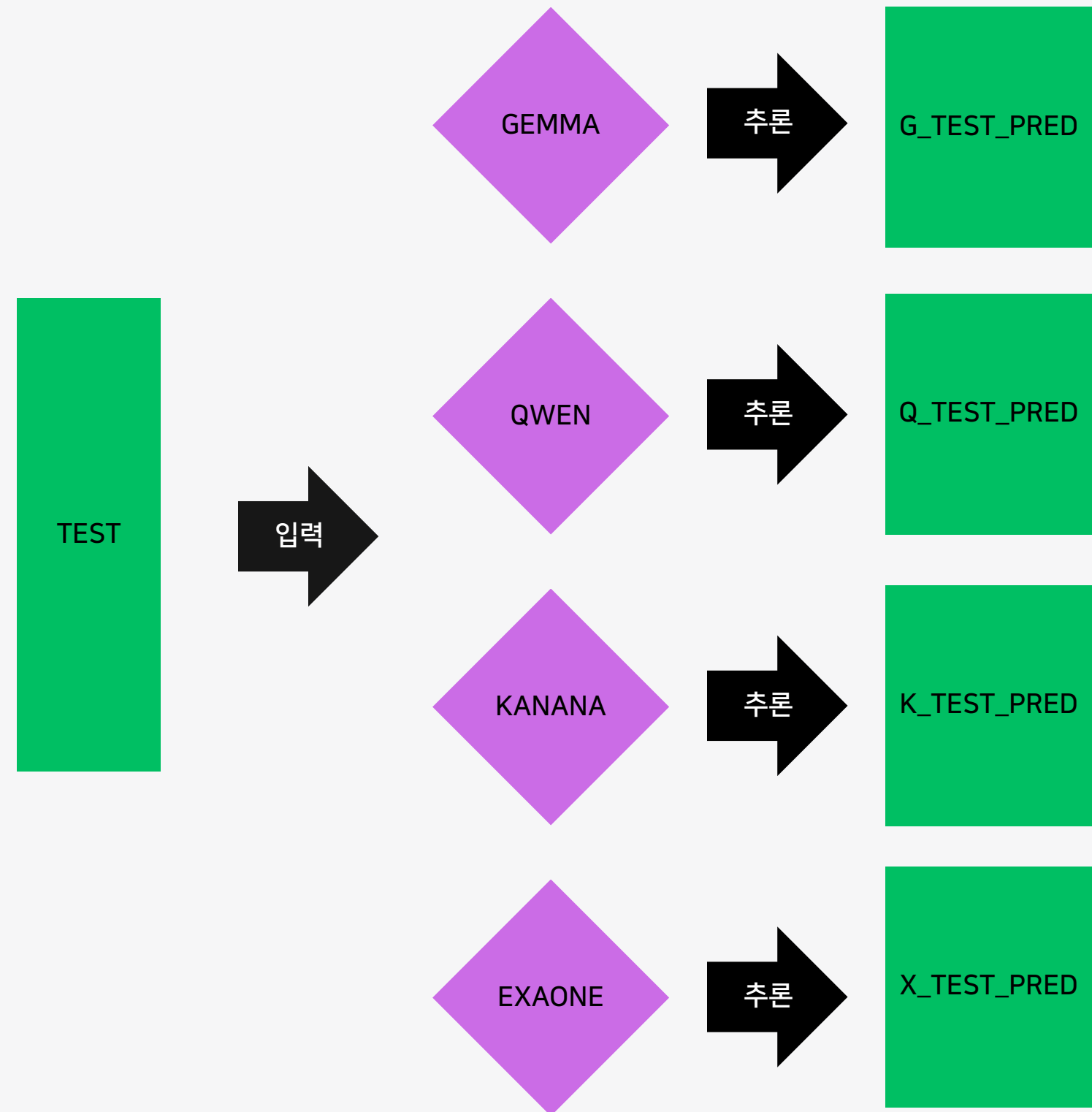
STEP1: 4개 BASE MODEL 학습 후 VAL 추론 결과 병합하여 META_TRAIN 생성



[meta_train]

ID	G_val pred	Q_val pred	K_val pred	X_val pred	Ground truth label
1					1
2					0
3					1
4					1
.					.
.					.

STEP2: 모델 별 TEST 추론 결과 병합하여 META_TEST 생성



[meta_test]

ID	G_test pred	Q_test pred	K_test pred	X_test pred
1				
2				
3				
4				
.				
.				

4-MODEL BLENDING ENSEMBLE

STEP3: META_TRAIN과 META_TEST로 META MODEL 학습 및 추론

